

Task 1

a)

Base info:

$P(S) = 0.7$, $P(A) = 0.3$ | S = Strawberry, A = Anchovy $P(\text{Round} | S) = 0.8$, $P(\text{Square} | A) = 0.9$ $P(\text{Red} | S) = 0.8$, $P(\text{Brown} | A) = 0.9$

The network correctly representing $P(\text{Flavor}, \text{Wrapper}, \text{Shape})$ is the third Bayesian net, as the likelihood of both wrapper and shape is inherently dependant on the flavor of the candy to begin with.

The first and the 2nd BNs, give the following dependency suggestions: i: *Wrapper, Shape* -> *Flavor* ii: *Wrapper* -> *Shape* -> *Flavor*, *Wrapper* -> *Flavor* Neither of these suggestions can accurately describe the actual dependancies: *Flavour* -> *Wrapper* & *Flavour* -> *Shape* = *Flavour* -> (*Wrapper, Shape*)

b)

The best model would again be the 3rd BN, as it accurately represent the causal relationship in the world as well as yielding a simpler CPT with fewer dependencies. The latter coming from the nature of compactness for CPTs: A CPT for boolean X_i with k boolean parents has 2^k rows for the combinations of parents values.

c)

Network i assert that Wrapper is indeed independent of Shape. This can easily be seen by creating the ancestral graph(no connections between Wrapper and Shape). However, network i also assert that Wrapper is conditionally independent of Shape, given Flavour.

d)

$P(\text{Shape}|\text{Flavor})$

$P(\text{Round} \text{Strawberry})$	$P(\text{Round} \text{Anchovy})$	0.8	0.1
$P(\text{Square} \text{Strawberry})$	$P(\text{Square} \text{Anchovy})$	0.2	0.9

$P(\text{Wrapper}|\text{Flavor})$

P(Red Strawberry)	P(Red Anchovy)	0.8	0.1
P(Brown Strawberry)	P(Brown Anchovy)	0.2	0.9

CPT: P(Shape, Wrapper | Flavour)

P(Strawberry)		P(Anchovy)		P(Strawberry)		P(Anchovy)	
P(Round Strawberry)	P(Square Strawberry)	P(Round Anchovy)	P(Square Anchovy)	0.8	0.2	0.1	0.9
P(Red Strawberry)	P(Brown Strawberry)	P(Red Anchovy)	P(Brown Anchovy)	0.8	0.2	0.1	0.9

$$P(\text{red}) = P(\text{Red}/S) * P(S) + P(\text{Red}/A) * P(A) = 0.8 * 0.7 + 0.1 * 0.3 = 0.59$$

d)

$$\text{Using Bayes: } P(S | \text{Red}) = P(\text{Red}/S) * P(S) / P(\text{Red}) = 0.8 * 0.7 / 0.59 = 0.949$$

f)

U(A): a, U(S): s

$$E[U(\text{candy})] = \sum_i P(\text{Flavour}_i) * U(\text{Flavour}_i) = P(S) * s + P(A) * a$$

Task 2

Exp Utility function: $U(x) = -e^{-x/R}$

a)

Base info:

$R = 500$, $U(\text{loss}) = U(0) = -1$ Option 1: $P(\text{win}) = 1.0$ and $U(500) = -0.368$, Option 2: $P(\text{win}) = 0.6$ and $U(5000) = -4.54 * 10^{-5}$

MEU principle: A rational agent must choose the action that maximizes expected utility. $EU(\text{Option 1}) < - P(\text{win}) * U(\text{win}) + P(\text{Loss}) * U(\text{Loss}) = 1 * U(500) + 0 * U(\text{loss}) = -0.368$ $EU(\text{Option 2}) < - P(\text{win}) * U(\text{win}) + P(\text{Loss}) * U(\text{Loss}) = 0.6 * U(5000) + 0.4 * U(\text{loss}) = -0.4$

$EU(\text{Option 1}) > EU(\text{Option 2}) \Rightarrow$ Assuming Mary is rational, should choose option 1.

b)

Base info:

$R = ?$, $U(\text{loss}) = U(0) = -1$ Want indifference between option A and option B:
 $EU(A) \sim EU(L_B) \mid L_B = [0.5, 500; 0.5, 0]$

Equation to solve for R: $-e^{-100/R} = -e^{-500/R} \cdot 0.5 + (-1) \cdot 0.5$ Solved for R using
`scipy.optimize.fsolve()`

In []:

```
import numpy as np
import scipy.optimize as so

def expected_utility_equation(R):
    # Certain amount utility (U(100))
    U_certain = -np.exp(-100 / R)

    # Lottery expected utility (0.5 * U(500) + 0.5 * U(0))
    U_lottery = 0.5 * (-np.exp(-500 / R)) + 0.5 * (-np.exp(0))

    # Indifference condition: EU(certain) = EU(lottery)
    return U_certain - U_lottery

R_initial_guess = 50
R_solution_corrected = so.fsolve(expected_utility_equation, R_initial_guess)
R_solution_corrected_rounded = round(R_solution_corrected, 3)

print(f'Numerical solution to R: {R_solution_corrected_rounded}')
```

Numerical solution to R: 152.38